

INFORMATION SECURITY

Assignment 3

Zunaira Abdul Aziz | BSSE23058 | Section A

Question 1:

1 a)

1. **Trojan Horse:** The intern downloaded a seemingly useful tool (3D Model Optimizer) that worked as expected but had hidden malicious effects (slowing the computer). A Trojan is defined as an apparently useful program containing additional malicious functionality.
2. **Worm:** The script spread to five other computers on the same network without those users downloading the tool. This matches the definition of a Worm, which propagates copies of itself to other computers in a network without any user action.
3. **Ransomware:** The server was locked, and a payment in Bitcoin was demanded to recover blueprints. This is Ransomware, which blocks access to data and asks for a ransom payment.
4. **Bootkit:** The malware re-infected the system within minutes of booting, even before user login or OS restoration could fully clean it. This indicates it was residing in the master boot record.

1 b)

Specific Type: Bootkit.

Mechanism: It manages to load before the operating system by infecting the master boot record (MBR) or the boot sector of the disk. Because it loads during the initial boot process, it gains control before the OS security features are active.

1 c)

Classification: Zombie (or Bot).

If the infected computers are used to launch a distributed attack (such as a DDoS attack) on another target, they are classified as Zombies. A collection of such compromised machines controlled by a central attacker is known as a Botnet.

Question 2:

2 a)

- **Input String:** ' OR 1=1 --.
- **Full Resulting SQL Query:**
SELECT name, salary FROM employees WHERE empID = " OR 1=1 --' AND status = 'active';
- This terminates the original empID string. The OR 1=1 is a tautology that always evaluates to True, forcing the WHERE clause to match every row. The -- is a SQL comment mark that nullifies the remainder of the query, including the status = 'active' requirement.

2 b)

- **Input String:** 101; UPDATE salary SET salary = 250000 WHERE empID = 999; --
- **Full Resulting SQL Query:** SELECT name, salary FROM employees WHERE empID = '101';
UPDATE salary SET salary = 250000 WHERE empID = 999; --' AND status = 'active';
- This uses a semicolon (;) to separate the legitimate SELECT command from the malicious UPDATE command, piggy-backing the second query on the first.

2 c)

1. **Security through Obscurity:** Hiding a page will not stop an attacker because automated bots and scanning tools do not rely on visual links; they scan for vulnerabilities, ports, and hidden forms systematically.
2. **Most Effective Architectural Change:** The single most effective change is the use of Prepared Statements (Parameterized Queries). This separates the SQL command logic from the data, ensuring that user input is never interpreted as executable code. (Note: While not explicitly named architectural in every slide, Defensive coding via stronger data validation and sanitizing inputs are the primary countermeasures listed) .

Question 3:

3 a)

Type of Attack: This is a Tautology-based SQL Injection attack (a form of In-band SQLi).

Result: The database will return the title and author of every book in the library, regardless of their category or whether they are currently available.

The input ' OR 1=1; -- performs three actions:

- The first quote (') closes the string for the category field.
- OR 1=1 is a tautology (a statement that is always true), which forces the WHERE clause to evaluate to true for every row in the table.
- The ; -- terminates the first command and comments out the rest of the original query (AND status = 'available'), ensuring the available restriction is ignored.

3 b)

Technique: The attacker would use Blind SQL Injection (specifically Inference).

How it differs:

In-band Attack: The attacker receives the results of the malicious query directly in the web page's output (e.g. seeing the list of books on the screen).

Blind/Inferential SQLi: There is no actual data transferred via the web application. Instead, the attacker asks the server true/false questions and observes the behavior (such as different page responses or time delays) to reconstruct information about the database version or table names.

3 c)

Based on the SQLi countermeasures discussed in the lectures, the developers should implement:

Sanitizing Inputs (Input Validation):

- **How it works:** This involves encoding all unsafe characters such as quotes ('), semicolons (;), and hyphens (--). By escaping these characters, the database treats them as literal text rather than command instructions.

Stronger Data Validation (Type Checking & Pattern Matching):

- **How it works:** The system should check that the input matches the expected format. For example, if a category is expected to be a string from a predefined list, any input containing SQL keywords or symbols should be rejected. Pattern matching ensures that the input distinguishes normal input from abnormal.

Question 4:

4 a)

The attacker can determine the exact value by using the relationship between the aggregate sum and the mean.

- **Step 1:** Calculate the total heart rate of the original 4 people: $\text{Sum} = \text{Average} \times 4$.
- **Step 2:** Query the new average for the group of 5 and calculate the new total: $\text{Sum} = \text{Average} \times 5$.
- **Step 3:** Subtract the first sum from the second: $\text{Heart Rate} = \text{Sum of 5} - \text{Sum of 4}$.
This is a classic Inference attack where information can be derived from a database without direct access by observing how aggregate results change when a specific record is added.

4 b)

Even if the threshold (k) is set to 5 (meaning a query must involve at least 5 people to run), a persistent attacker can bypass this using a Tracker.

A tracker is a padding set of records that the attacker already knows. By querying (Tracker +

SubjectX) and then querying the Tracker alone, the attacker can still isolate Subject X's data. If both queries meet the size requirement (e.g. 5 or more people), the restriction fails to stop the subtraction logic.

4 c)

Comparison: Data Perturbation adds noise directly to the database records themselves. Output Perturbation leaves the records intact but adds noise to the *result* of the query before it is displayed.

Effectiveness for Research: Output Perturbation is more effective in this scenario.

Reasoning: Since the researchers require the original records to remain 100% accurate for internal use, they cannot use Data Perturbation (which modifies the actual stored data). Output Perturbation allows the internal database to stay accurate while protecting privacy for public aggregate queries.

Question 5:

5 a)

If the Current_Balance column is encrypted using Field-Level Encryption, the primary challenge is performing range queries or comparisons (like Greater than \$5,000).

When data is encrypted at the field level, it is stored as ciphertext. Standard database management systems cannot perform mathematical operations or logical comparisons (e.g. >, <) on ciphertext because the encrypted values do not preserve the numerical order of the original data. To run this query, the application would typically have to decrypt the entire column into memory, which is computationally expensive and poses a security risk.

5 b)

In Record-Level Encryption, the entire row is treated as a single block. To find a Customer_Name efficiently, the system uses Attribute Indexes based on Partitions:

- **Mechanism:** The range of possible values for an attribute (like names) is divided into non-overlapping partitions. For example, names starting with A-D are in Partition 1, E-H in Partition 2, etc.
- **How it helps:** Each encrypted record is stored with an unencrypted index value representing its partition. If a user searches for John, the database identifies that J falls into Partition 3. It then only retrieves and decrypts the records associated with Partition 3, rather than the entire database. This significantly reduces the processing load.

5 c)

Database Encryption is called the Last Line of Defense because it is the final layer of protection. If an attacker has managed to bypass all exterior security, the encryption ensures that the data

remains unreadable and useless to them.

Two other security layers that should be compromised first:

- **Access Control / Authentication:** The attacker must first bypass the login mechanisms (usernames, passwords, or multi-factor authentication) that verify who is allowed to access the system.
 - **Firewalls / Network Security:** The attacker must penetrate the network perimeter or the firewall rules that restrict traffic to the database server.
-

Question 6:

6 a)

Exact Input String: ' OR 1=1 --

Full Resulting SQL Query: SELECT name, salary, position FROM payroll_data WHERE eid = " OR 1=1 -- ' AND status = 'active';

1. The first quote (') closes the empty string for the eid field.
2. The OR 1=1 is a tautology (a condition that is always true), which forces the WHERE clause to evaluate to true for every single row in the table.
3. The double hyphen (--) is the SQL comment symbol. This nullifies the AND status = 'active' requirement by turning the rest of the original query into a comment, meaning the database ignores it entirely.

6 b)

Exact Input String: 101'; UPDATE payroll_data SET salary = 250000 WHERE eid = '999'; --

Full Resulting SQL Query:

SELECT name, salary, position FROM payroll_data WHERE eid = '101'; UPDATE payroll_data SET salary = 250000 WHERE eid = '999'; -- ' AND status = 'active';

This attack uses a semicolon (;) to terminate the first legitimate SELECT command and piggyback a second, malicious UPDATE command.

The 101' properly closes the first string, and the -- at the end ensures that the trailing syntax from the original PHP code does not cause a syntax error.

6 c)

Security through Obscurity: According to the Inference and SQLi attempts slides, hiding a page is ineffective because automated bots and scanning tools do not browse a site like a human user. They systematically scan for vulnerabilities, ports, and hidden forms. A real-world example in the slides shows that even if a form is hidden from actual users, bots will still find it and submit malicious payloads (like UNION ALL SELECT NULL# or WAITFOR DELAY) to probe

for weaknesses.

Most Effective Architectural Change:

The single most effective architectural change is the use of Prepared Statements (Parameterized Queries). How it works: Instead of building a query string with user input, the SQL command is sent to the database first with placeholders (e.g., WHERE eid = ?). The user input is then sent separately as a parameter. The database treats the parameter strictly as data and never as executable code, making it impossible for a tautology or a semicolon to change the query's logic.